



University
of Glasgow

THE TIMES
THE SUNDAY TIMES

GOOD
UNIVERSITY
GUIDE
2022

SCOTTISH
UNIVERSITY
OF THE YEAR

Dr Si Chen

si.chen@glasgow.ac.uk

More Functions & Objects - EN2083

WORLD
CHANGING
GLASGOW

<https://moodle.gla.ac.uk/mod/attendance/password.php?session=643877>



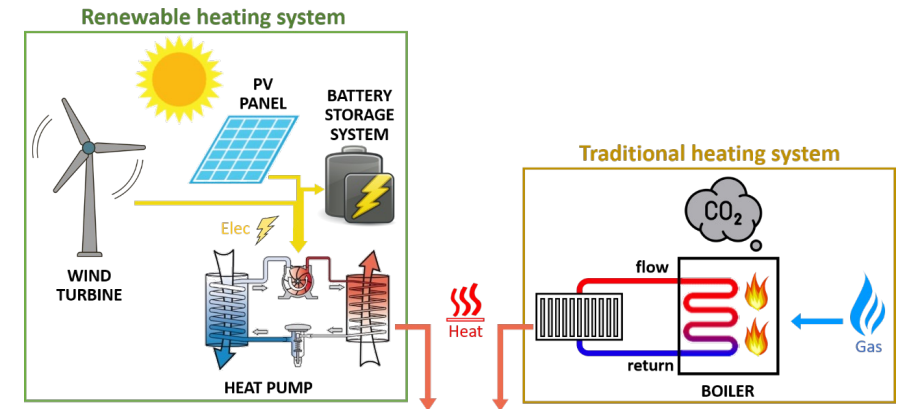
University
of Glasgow

My background

PhD in Electronics & Electrical Engineering

MSc in Risk and Uncertainty

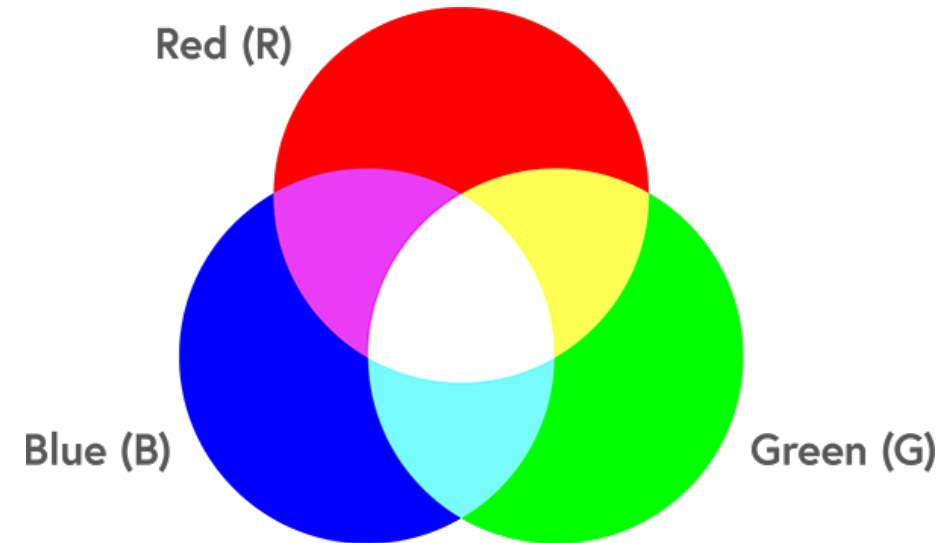
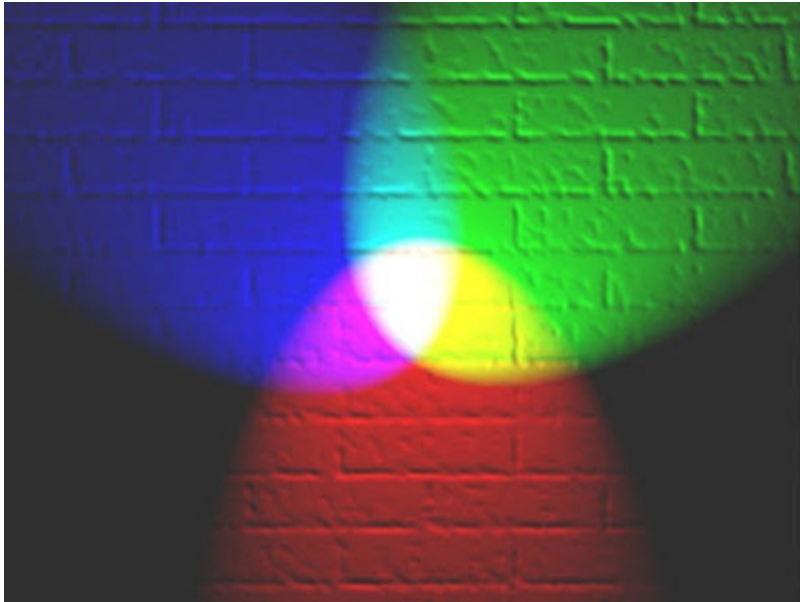
BEng in Electrical Engineering and Automation



University Campus



Example – Image Process



Colour Image

=



Red

+



Green

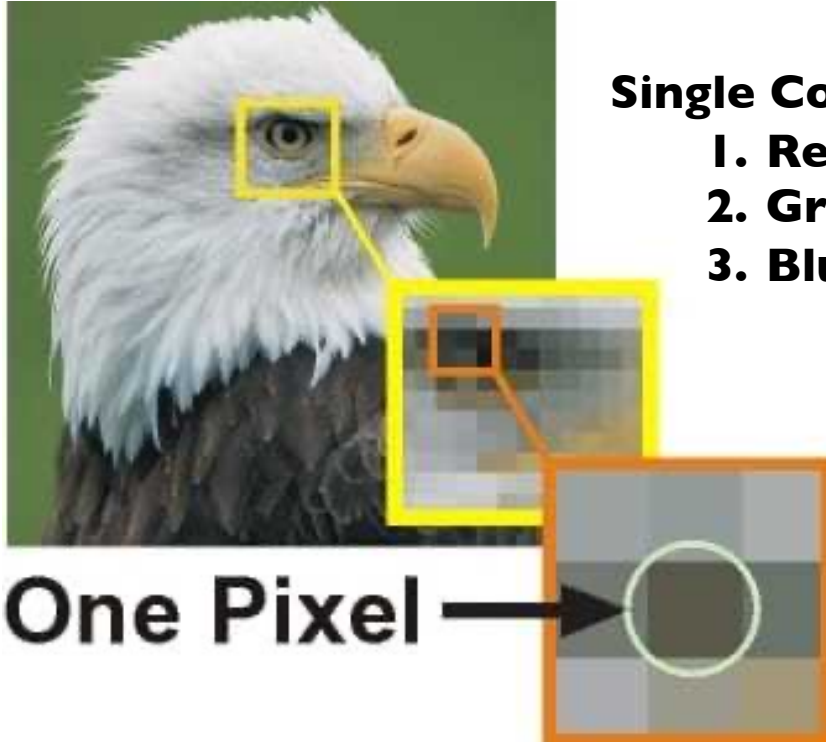
+



Blue

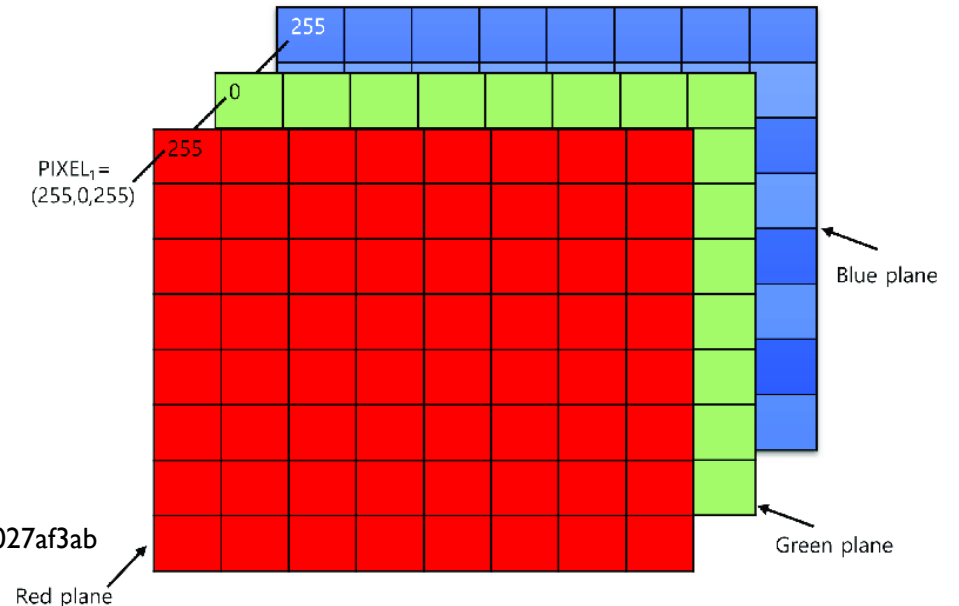
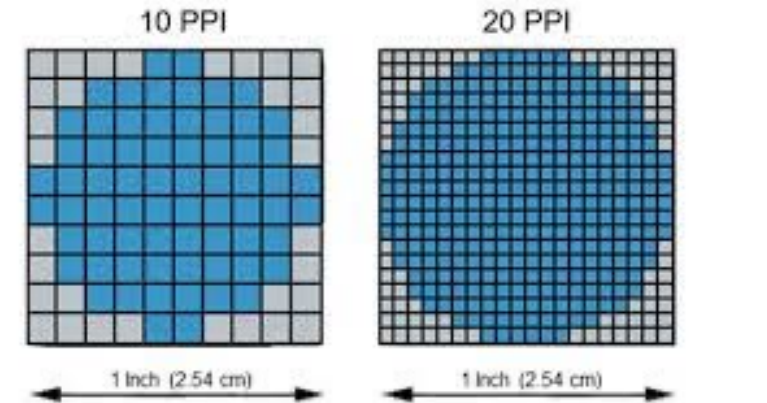


Example – Image Process



Single Colors (255 for only one colour):

1. **Red:** (255, 0, 0)
2. **Green:** (0, 255, 0)
3. **Blue:** (0, 0, 255)



Recap – Import Package

- modules are just python files with functions in them.
- import them into the top of your code in a few simple ways (to avoid duplicates).
- work the same way as modules, but usually package functions are more serious beasts!
- aliasing to reduce typing:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

- aliasing one particular function

```
from math import sqrt as r
```



Recap – Structure

- a 'type' (like a float) just describes a variable which can contain a number. You then need to bring the variable into existence.
- a 'Class' just describes an Object which can contain data and functions. You then need to bring the Object into existence.
- functions acting on the object are described in the Class, and refer to object 'themselves' and then the objects can have the functions applied to them
- data about each object is described in the Class, again referring to the object 'itself' and then the objects can access that data
- in classes, there is a function, `__init__`, which you write, automatically called when an object is created - the constructor.
- It sets up the initial variables of the object, either itself, or from data supplied



Recap – Define function

- functions are defined before use, and then called
- every function (just like in maths) returns one thing, using the return statement
- even if you don't return, you actually do, you return with a value of None.
- if you want a *function* to muck around with class data at class level (not object level)



Task 1 – Define an object

- Picture

- Shape (pixels) – Width & Height
- Data of each colour – RGB value

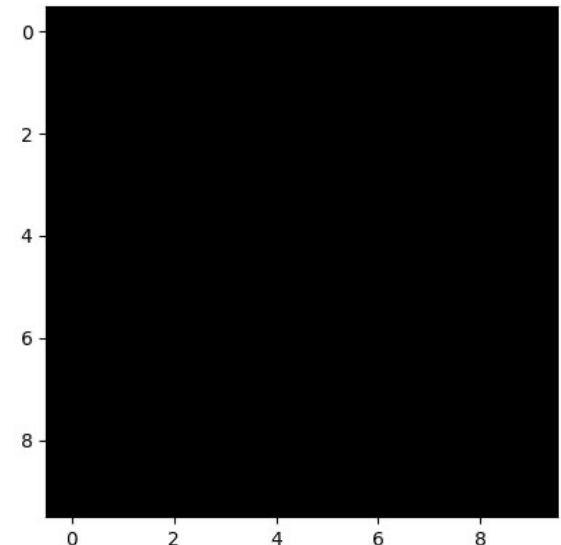
How can we use class
to define an image?

```
import numpy as np
class image():
    def __init__(self, width, height):
        self.width = width;
        self.height = height;
        self.shape = (width,height);
        self.R = np.zeros(self.shape);
        self.G = np.zeros(self.shape);
        self.B = np.zeros(self.shape);
```

Task I – Define an object

How can we use class
function to display an
image?

```
class image():  
    ...  
    def show(self):  
        image = np.dstack([self.R, self.G, self.B]);  
        plt.imshow(image);  
  
img1 = image(10, 10);  
img1.show();
```



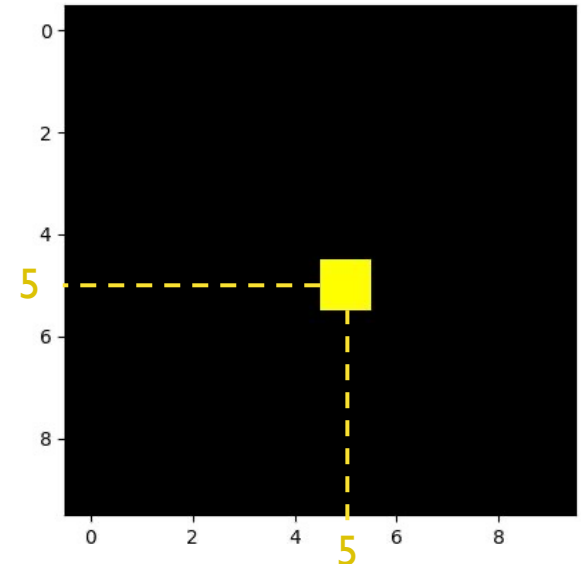
Task I – Define an object

How can we use class function to display an image?

```
class image():  
    ...  
    def show(self):  
        image = np.dstack([self.R, self.G, self.B]);  
        plt.imshow(image);
```

```
img1 = image(10, 10);  
img1.R[5, 5] = 255;  
img1.G[5, 5] = 255;  
img1.B[5, 5] = 0; } RGB(255, 255, 0)  
img1.show();      → YELLOW
```

We can manipulate the colour of any pixel!



Recap - Read data from file

- to get a whole file in one go

```
myfilename = '/home/data/levels.txt'      # file path
with open(myfilename, 'r') as f:          # open for read
    contents = f.read()                    # might overflow !

print(contents)                           # firehose of data
```

- and line by line...

```
with open(filename, 'r') as f:
    for line in f:
        print(line.rstrip())
```

- here `rstrip` gets rid of extra end-of-line characters added by python



Recap - Read data from file

- but you probably wanted to store the lines

```
with open(filename) as f:  
    lines = f.readlines()                # store in list
```

```
for line in lines:  
    print(line.rstrip())                # use stored list  
# controlled firehose of data
```

- however, to do any more you need to know the precise nature of the data in advance

```
data = []  
for line in lines:  
    data.append(float(line))            # each line one float
```



Task II – Read an image

```
import matplotlib.pyplot as plt

class image():
    ...
    def copy(self, image):
        self.R = image[:, :, 0];
        self.G = image[:, :, 1];
        self.B = image[:, :, 2];

filename = 'main_building.jpg';
original_image = plt.imread(filename);
img_width = original_image.shape[0];
img_height = original_image.shape[1];

my_img = image(img_width, img_height);
my_img.copy(original_image);
my_img.show();
```



Image source: <https://www.universitystory.gla.ac.uk>



Recap – Operators

- four basic: $+$ $-$ $/$ $*$
- integer division and modulo operator: $//$ $\%$
- always use brackets if even vaguely unsure.
- works the same way as normal mathematics where $*$ $/$ $\%$ take precedence over $+-$.
- Within each group precedence is typically ***left to right***.



University
of Glasgow

Task III – Inverse Colour

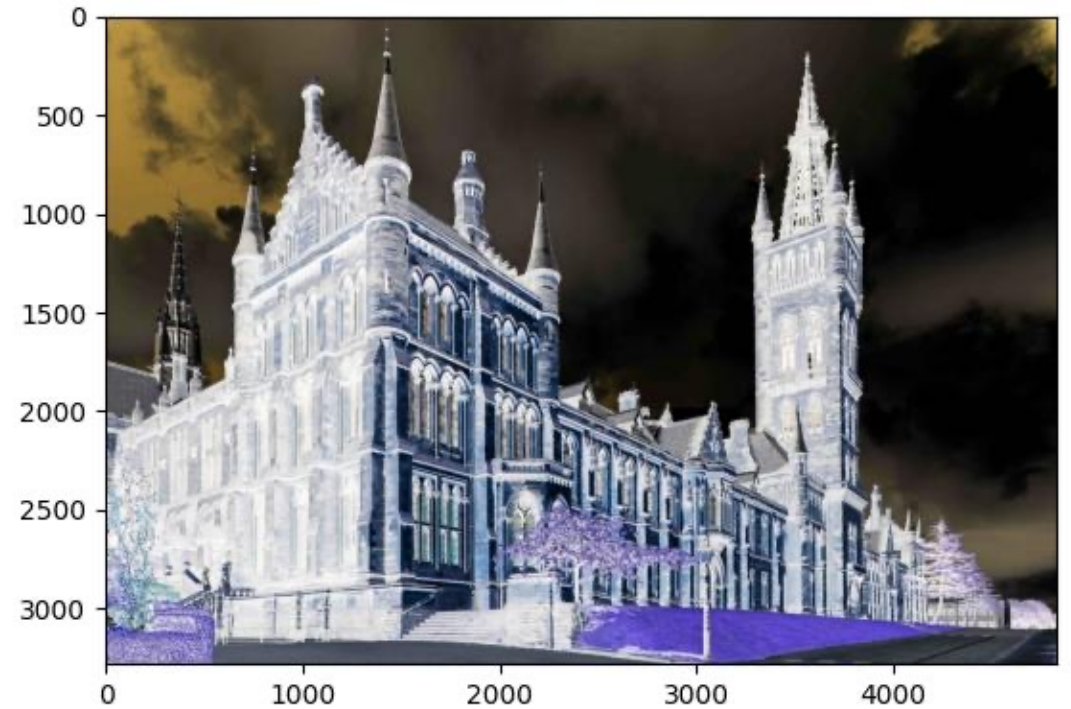
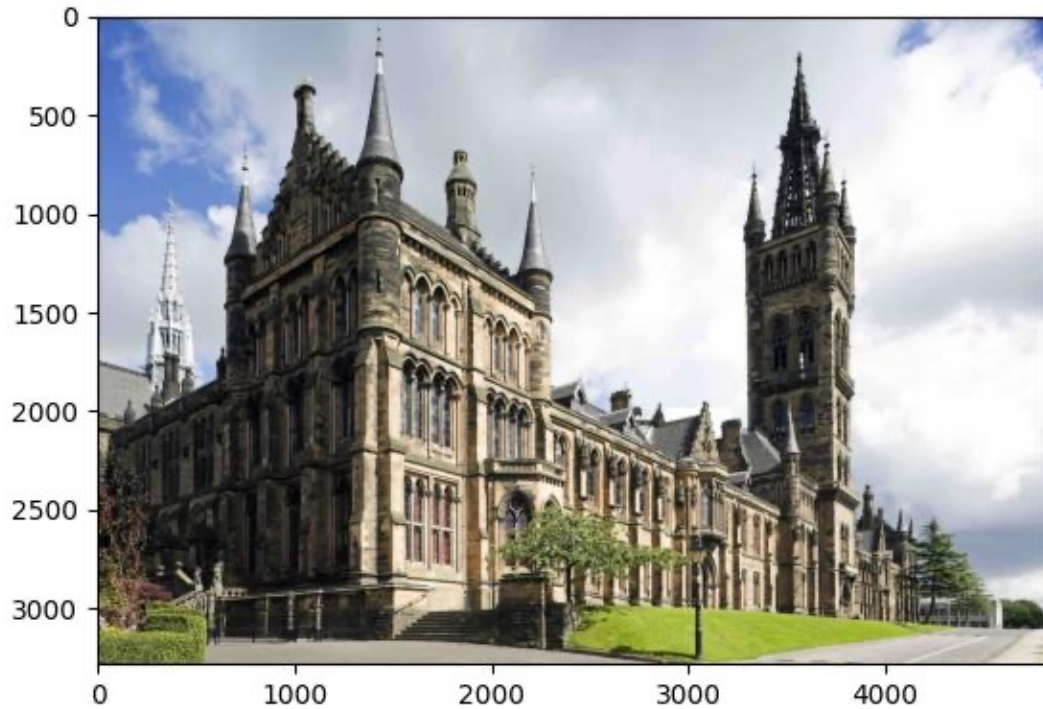


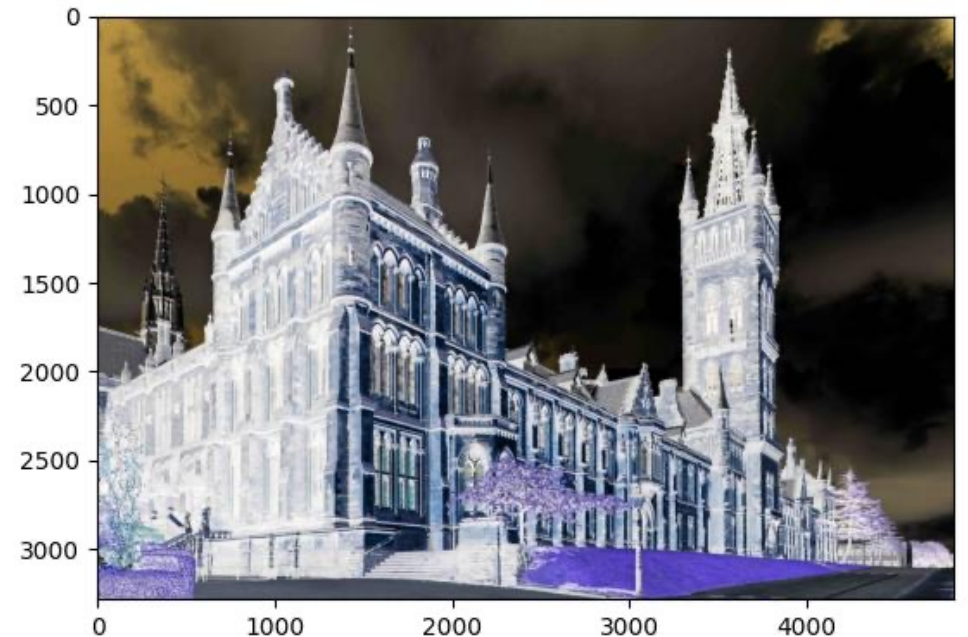
Image source: <https://www.universitystory.gla.ac.uk>



University
of Glasgow

Task III – Inverse Colour

```
class image():  
    ...  
    def inverse(self):  
        image_R = 255 - self.R;  
        image_G = 255 - self.G;  
        image_B = 255 - self.B;  
        image = np.dstack([image_R, image_G, image_B]);  
        plt.imshow(image);  
  
my_img.inverse();
```





Recap – Variable type

- Python works out the variable type from how it's used.
- What if you *want* another type. Need inbuilt conversion functions.
- `int("15")` is 15
- `int("15.7")` is 15
- `int("3f", 16)` is 63 (translate from hex)
- `float("-11.28E1")` is -112.8
- `round("15.11", 0)` is 15
- `round("15.11", 1)` is 15.1
- `bool(x)` is True if x 'exists'



University
of Glasgow

Task IV – Grayscale

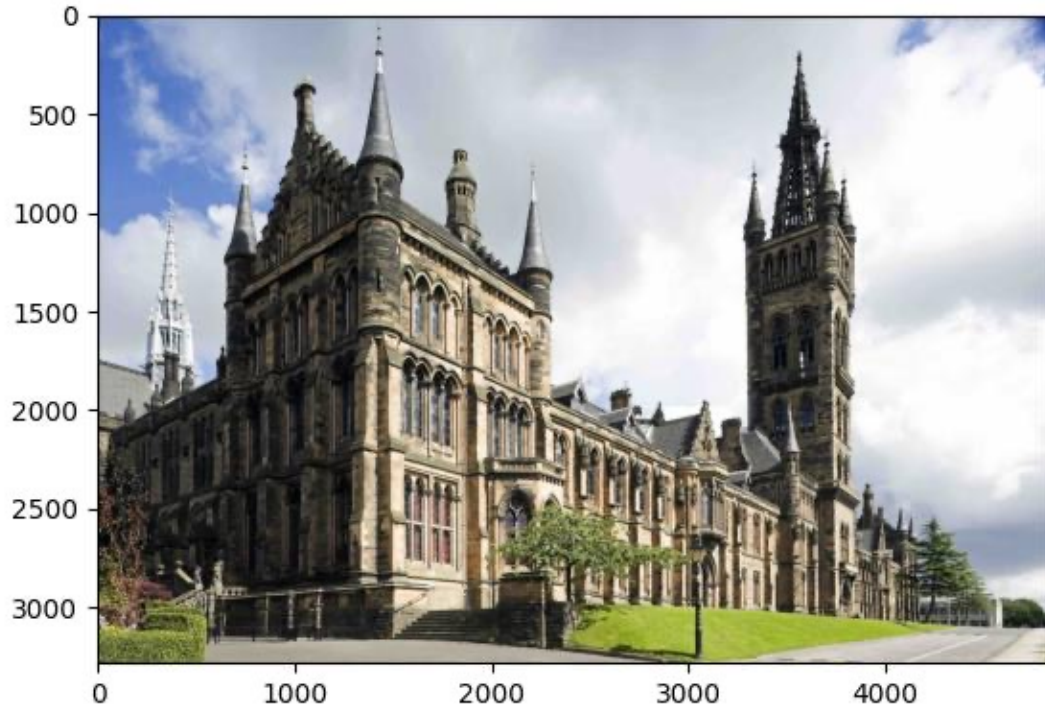


Image source: <https://www.universitystory.gla.ac.uk>

Task IV – Grayscale

```
class image():
```

```
...
```

```
def grey(self):
```

```
    image = np.dstack([self.R, self.R, self.R]);  
    plt.imshow(image);
```

The R,G,B colours are
set to the same.

```
my_img.grey();
```



We can use just R value to present all RGB colours.
But the information within G and B will be lost.



Task IV – Grayscale

```
class image():
```

```
...
```

```
def grey(self):
```

```
    → grey_color = (0.3*self.R + 0.59*self.G + 0.11*self.B).astype(int);  
    image = np.dstack([grey_color, grey_color, grey_color]);  
    plt.imshow(image);
```

Transfer data type to 'int'

```
my_img.grey();
```

- The RGB values are converted to grayscale using the NTSC formula:
$$0.299 \cdot \text{Red} + 0.587 \cdot \text{Green} + 0.114 \cdot \text{Blue}$$
- This formula closely represents the average person's relative perception of the brightness of red, green, and blue light.



Recap – *if* statements

❖ Selection statements take three forms:

- single selection

```
if (att < 75):  
    grade = "CR"
```

- double selection

```
if (marks < 40):  
    passed = 0  
else:  
    passed = 1
```

- indentation tells how much is `if` or `else`

- with multiple statements, the third type, the formatting is done like this:

```
if (<expression1>):  
    # statements indented by 4 spaces  
    # when <expression1> true  
  
elif (<expression2>):  
    # statements indented by 4 spaces  
    # when <exp2> true but <exp1> false  
  
else:  
    # statements indented by 4 spaces  
    # when both expressions false
```



University
of Glasgow

Task V – Vintage Style

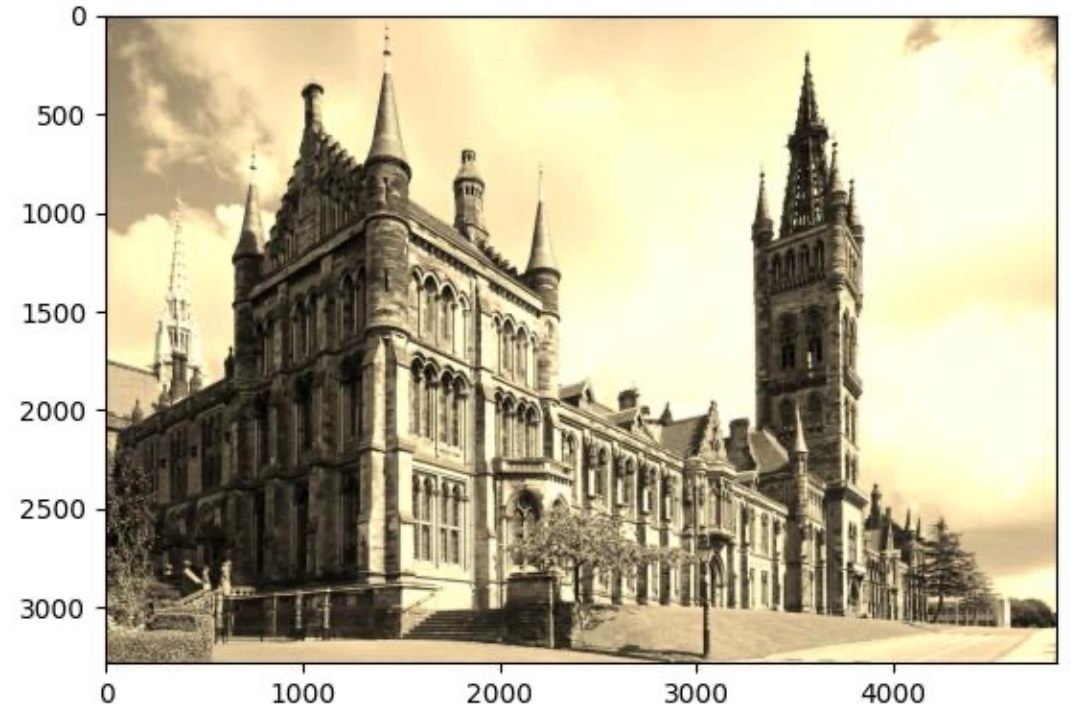
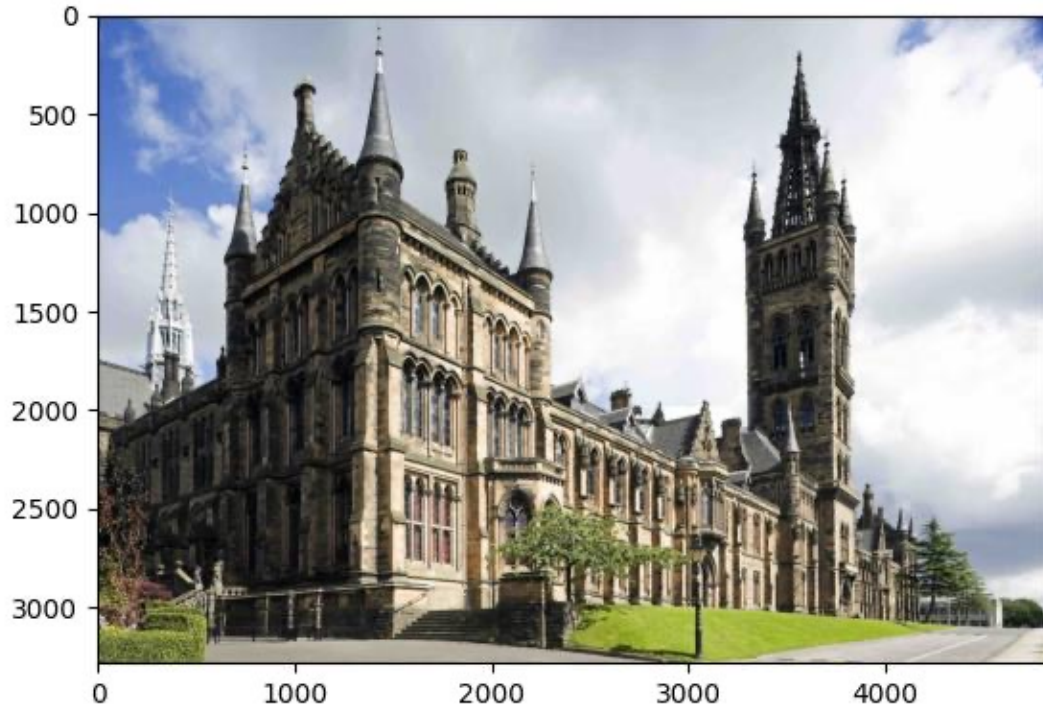


Image source: <https://www.universitystory.gla.ac.uk>

Task V – Vintage Style

```
class image():
```

```
...
```

```
def vintage(self):
```

```
    image_R = 0.393*self.R + 0.769*self.G + 0.189*self.B; ←
```

```
    image_G = 0.349*self.R + 0.686*self.G + 0.168*self.B; ←
```

```
    image_B = 0.272*self.R + 0.534*self.G + 0.131*self.B; ←
```

```
    image = np.dstack([image_R, image_G, image_B]);
```

```
    plt.imshow(image);
```

sepia toning

Could be greater than 255.

```
def clamp(n,minn,maxn):
```

```
    if n < minn:
```

```
        return minn
```

```
    elif n > maxn:
```

```
        return maxn
```

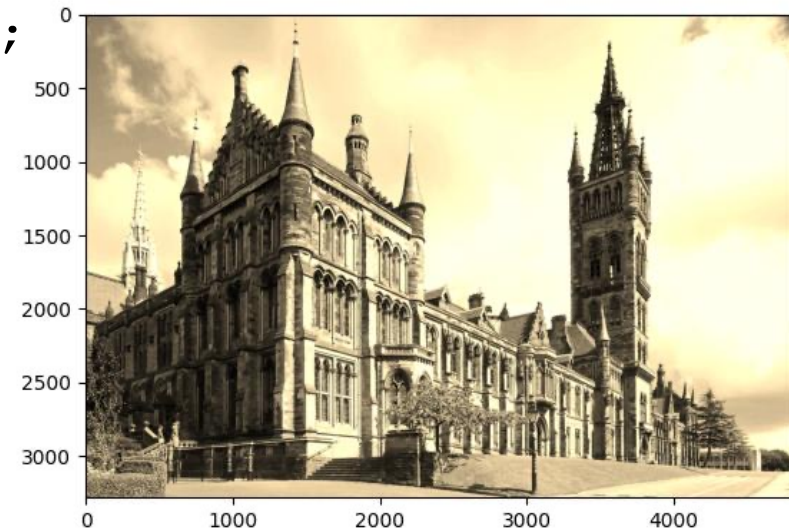
```
    else:
```

```
        return n
```

Use for loop to check the value of each pixel.

Task V – Vintage Style

```
class image():  
    ...  
    def vintage(self):  
        ...  
        for i in range(self.width):  
            for j in range(self.height):  
                image[i,j,0] = clamp(image_R[i,j], 0, 255);  
                image[i,j,1] = clamp(image_G[i,j], 0, 255);  
                image[i,j,2] = clamp(image_B[i,j], 0, 255);  
        ...  
        plt.imshow(image);  
  
my_img.vintage();
```



Recap – Loops

- Python has two loop/iteration constructs: `while`, and `for`

- the syntax of 'while' is :

```
while (<expression>) :
```

```
    <statement>    # can be lots!
```

- the computer evaluates the expression, and exits the loop if it is false.
- if the expression was true, the computer executes the statement(s) and then goes back to check the expression again.

- the classic counter loop in C, Pascal, etc :

```
for (i=0; i<10; i++) <statement>
```

- `i` is a loop counter, which counts from zero to 9 or 10, and increments by one each time (the `i++`).
- counting loops are ubiquitous in all programming languages - and big source of errors (is it 9 or 10?)



Recap – Loops

- `while` (and `for`) loops can 'abort' :
- `break`
aborts the whole set of all possible loops,
and moves on...
- `continue`
aborts this particular loop at the 'continue'
point, but goes back to see if more loops
need carried out.



University
of Glasgow

Task VI – Vignette Effect

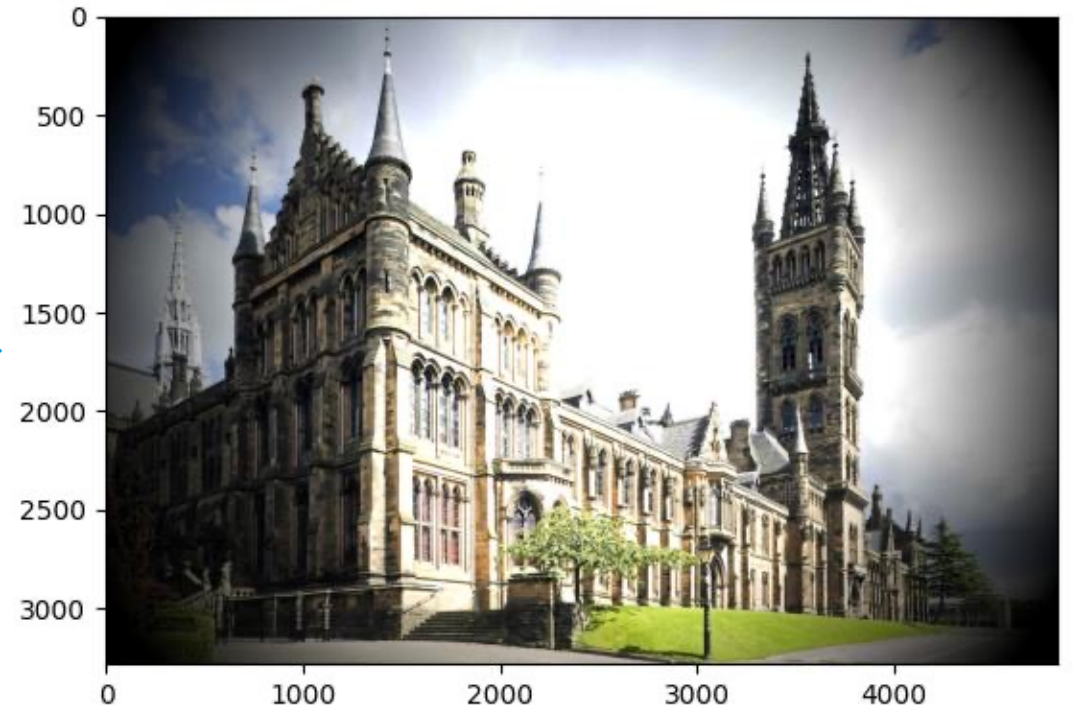
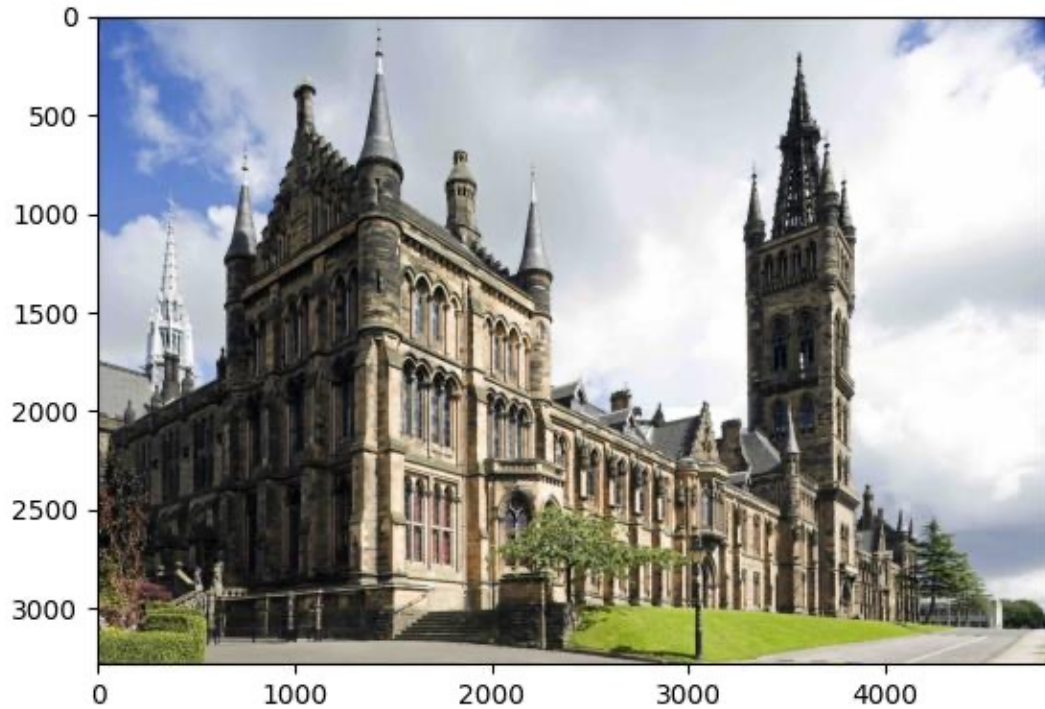
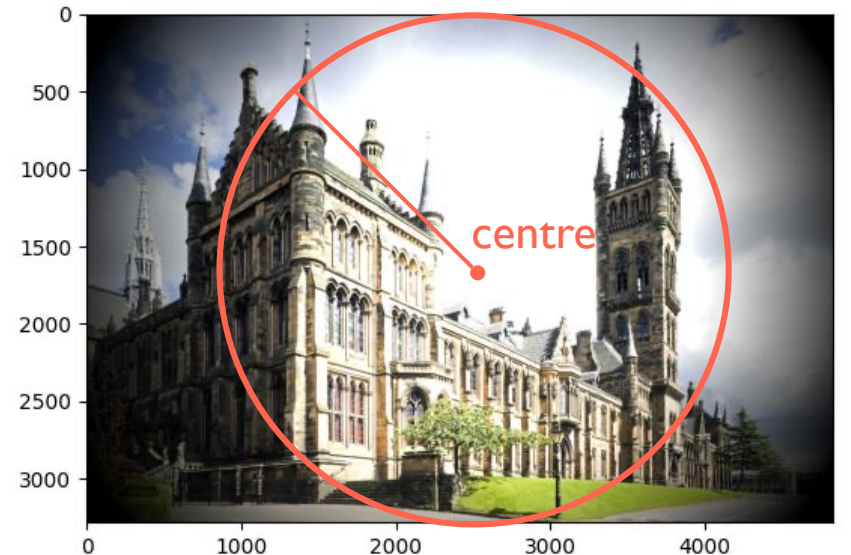


Image source: <https://www.universitystory.gla.ac.uk>



Task VI – Vignette Effect

```
class image():  
    ...  
    def Vignette(self):  
        centreX = self.width / 2;  
        centreY = self.height / 2;  
        radius = min(centreX, centreY);  
        image = np.dstack([self.R, self.G, self.B]);  
        ...
```



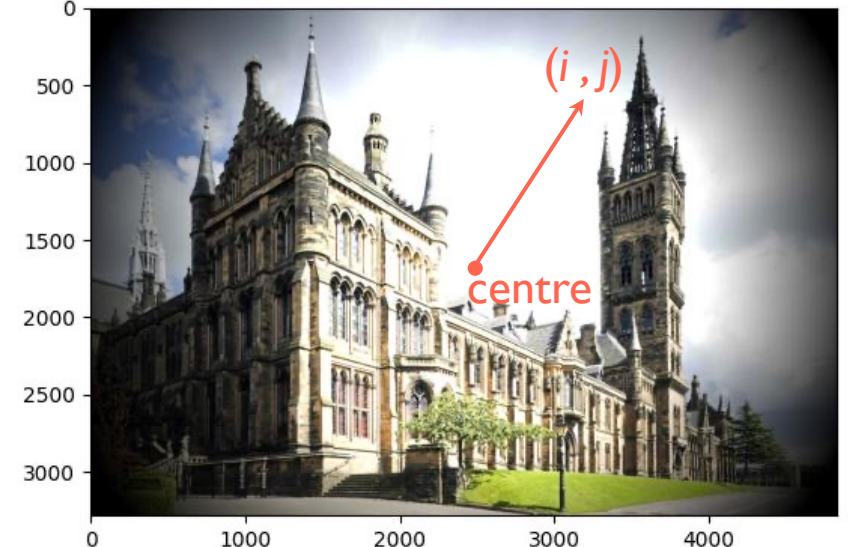
Task VI – Vignette Effect

```
...  
for i in range(self.width):  
    for j in range(self.height):  
        distance = np.sqrt((i - centreX)**2 + (j - centreY)**2);  
        gain = 1 + 1.5*(1 - distance/radius);  
        image[i,j,0] = int(clamp(self.R[i,j]*gain, 0, 255));  
        image[i,j,1] = int(clamp(self.G[i,j]*gain, 0, 255));  
        image[i,j,2] = int(clamp(self.B[i,j]*gain, 0, 255));  
plt.imshow(image);
```

The operator precedence? ←

my_img.Vignette();

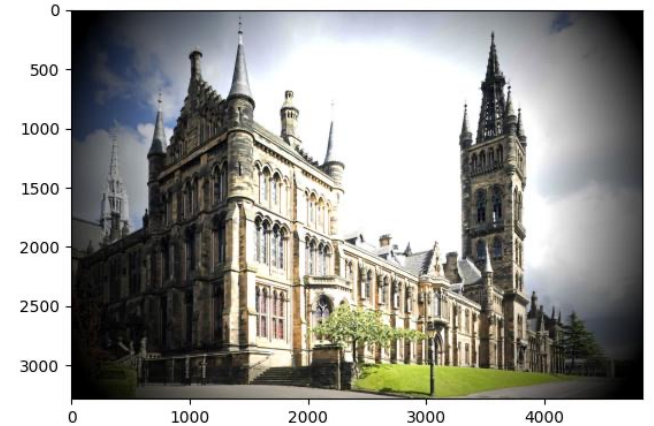
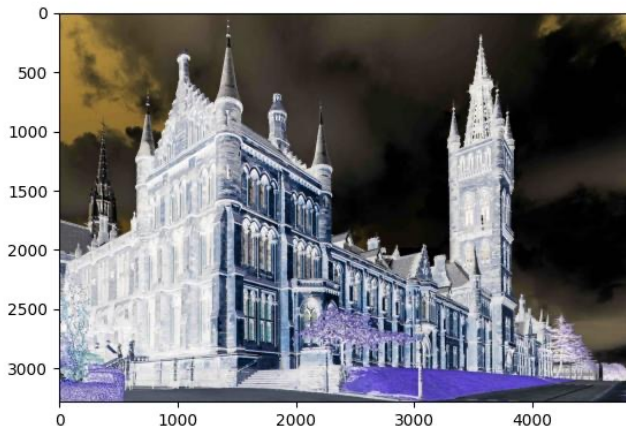
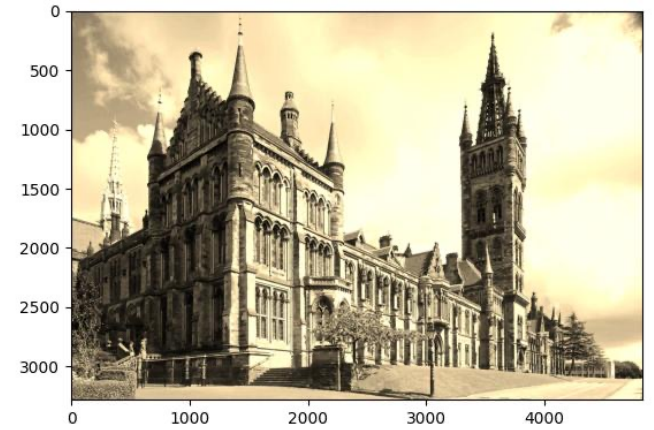
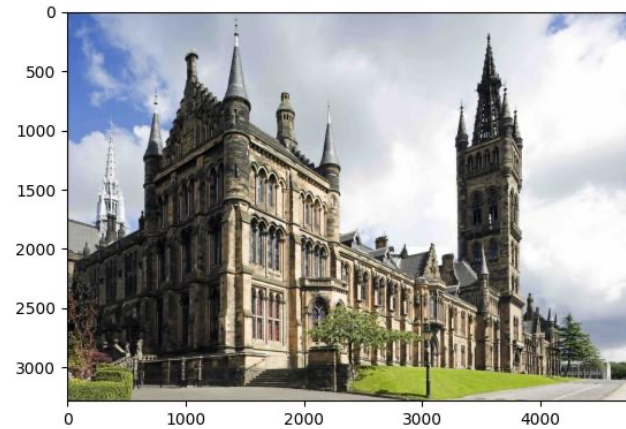
Increase or decrease the brightness of each pixel according to its distance to the centre.





University
of Glasgow

All Tasks Completed!



For more functions of image
processing, try OpenCV or
Pillow/PIL!



University
of Glasgow

Thank you

#UofGWorldChangers



@UofGlasgow

Recap class structure

- a 'type' (like a float) just describes a variable which can contain a number. You then need to bring the variable into existence.
`myHeight = 2.0`
- a 'Class' just describes an Object which can contain data and functions. You then need to bring the Object into existence.

```
class Car():  
    """Model a Car."""  
  
myCar = Car('nissan', 'micra')
```

Recap class structure

- in classes, there is a function, `__init__`, which you write, automatically called when an object is created - the constructor.
- It sets up the initial variables of the object, either itself, or from data supplied

```
def __init__(self, make, model):  
    """Initialise"""  
    self.make = make  
    self.model = model  
    self.tank_cap = 60      #60 litre tank  
    self.tank = 0           #initial fuel fill
```

Recap class structure

- definition :

```
class Car():
    """Model a Car."""
    def __init__(self, make, model):
        """Initialise"""
        self.make = make
        self.model = model
        self.tank_cap = 60      #60 litre tank
        self.tank = 0           #initial fuel fill

    def fillup(self):
        """fill tank to brim"""
        self.tank = self.tank_cap
        print("Full!")

    def ... #other functions
```

What can we do if we want to add a new class named **ElectricCar**?

Recap class structure

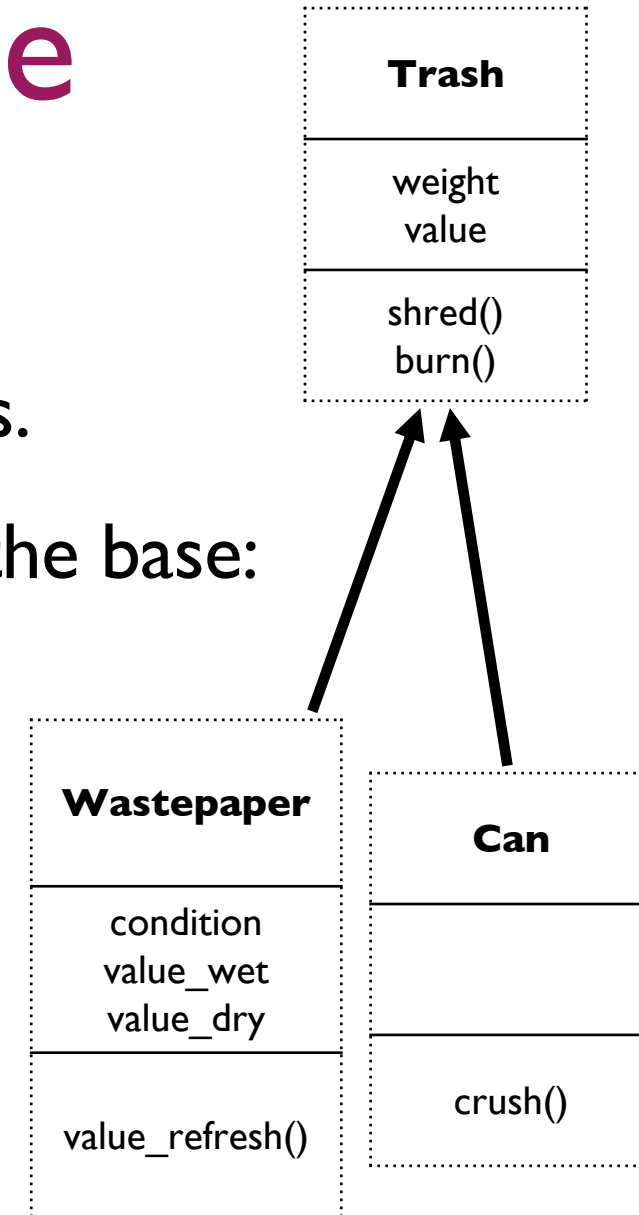
- What can we do if we want to add a new class named ElectricCar?
 - ✗ A) We can define a new class named 'ElectricCar' by defining all the same attributes and methods again. [make/model/...](#)
 - ✓ B) We can 'borrow' the attributes and methods from the traditional car class and make some changes. [Inheritance](#)

DRY Principle: **Don't**
Repeat
Yourself



Inheritance

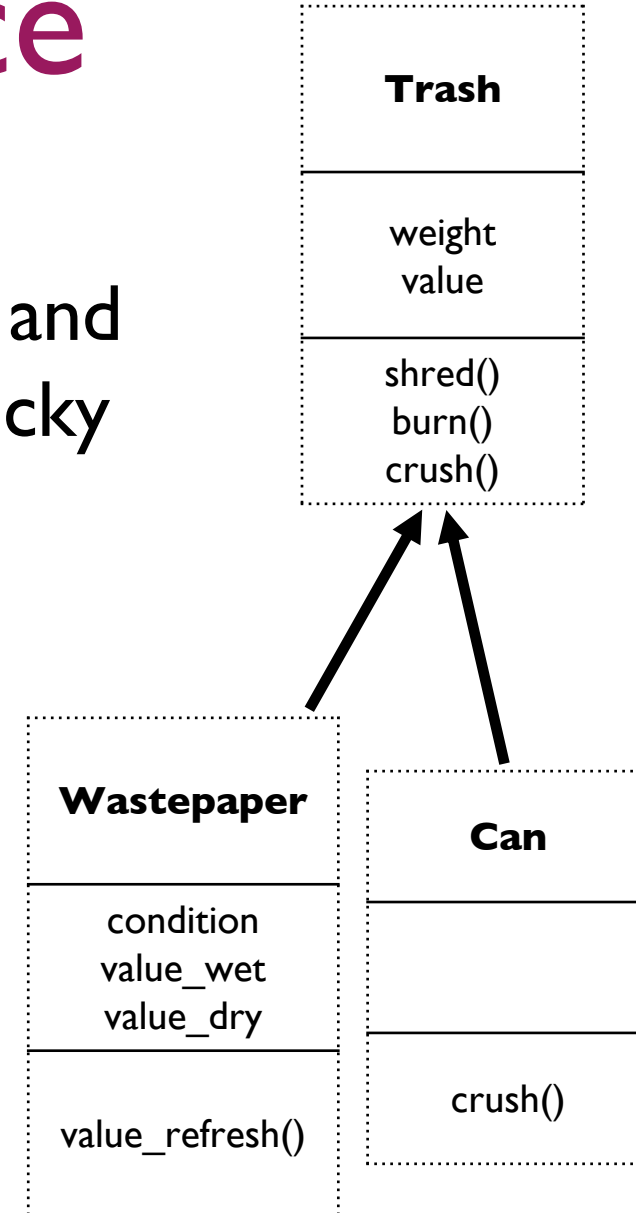
- Start from a base/super/parent type/class.
- Invent a new class, initially cloned from the base: an inherited/sub/child/derived class.
- Add new data and functions to the derived class, and override functions of base.





Inheritance

- The balance of what goes in the base class and how the derived classes augment, is the tricky design bit.
- Can decide *only* to override (a **Can** is a **Trash**)
- If you add stuff to the child it can do more (**Waste paper** is *like* **Trash**)





Inheritance

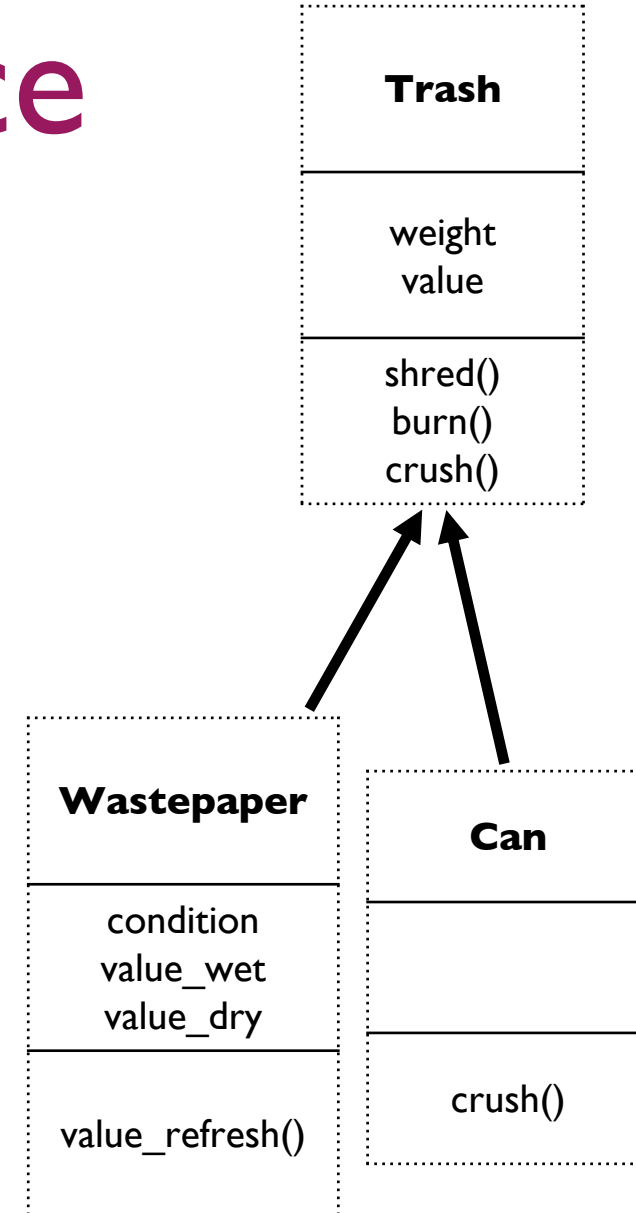
- If you are writing code,...

```
doStuff(x)
...
x.crush() ...
```

- this, amazingly, works !

```
Trash t; Wastepaper w; Can c;
```

```
doStuff(t) #base crush
doStuff(w) #base crush
doStuff(c) #specialised crush
```





Inheritance in cars

- class picks up Car, gets all data/methods :

```
class ElectricCar(Car):
```

- so can do all that the previous objects can do :

```
myOtherCar = ElectricCar('nissan', 'leaf')  
print(myOtherCar.make)  
print(myOtherCar.model)
```

- but !

```
myOtherCar.fillup() # oops, still works !  
print(myOtherCar.fuel_type) #still petrol
```

```
class Car():  
    """Model a Car."""  
    def __init__(self, make, model):  
        """Initialise"""  
        self.make = make  
        self.model = model  
        self.tank_cap = 60    #60 lit  
        self.tank = 0         #initia  
  
    def fillup(self):  
        """fill tank to brim"""  
        self.tank = self.tank_cap  
        print("Full!")
```

inheritance in cars

- We need to **override** methods from parent class :

```
def fillup(self):  
    """Not on an electric car."""  
    print("No fuel tank!")  
)
```

VS.

```
class Car():  
    """Model a Car."""  
    def fillup(self):  
        """fill tank to brim"""  
        self.tank = self.tank_cap  
        print("Full!")
```

- and need to **override** data from parent class !

```
fuel_type = 'electricity'
```

inheritance in cars

- We can define new attributes and methods in the derived class:

```
class ElectricCar(Car):  
    """Model an electric Car."""  
    def __init__(self, make, model):  
        """Initialise electric car"""  
        super().__init__(make, model):  
        self.battery_cap = 30    #30 kWh battery  
        self.charge = 0         #initial charge  
  
    def charge(self):  
        """Fully charge battery"""  
        self.charge = self.battery_cap  
        print("Full!")  
  
    def ... #other functions and private vars
```

Define new
attributes



“__init__” been
overridden !!

We want to extend,
but not override
“__init__”

- adds data and methods of own !

other classy info

- the data that a class defines, and which are therefore properties of objects, can be classes themselves.
- a car Class, and all the Car objects instantiated from it can contain data which are themselves objects (indeed in python, all the strings and numbers already are)
- you can of course have Lists of Cars, and Dictionaries with keyword, Car pairs !



other classy info

- Classes are great, but you end up with large text files of code. Usually these are stored separately as .py modules.
- So then you can do :

```
from car import Car, ElectricCar
```
- or :

```
import car    # file saved as car.py
```
- to import the module and start using the objects therein.

Basis of Inheritance

define a superclass

```
class super_class:
```

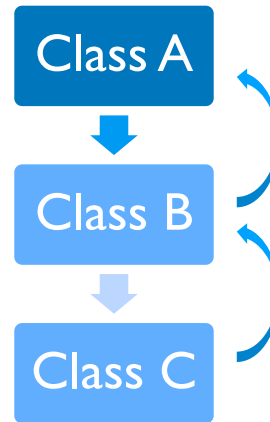
```
    # attributes and method definition
```

inheritance

```
class sub_class(super_class):
```

```
    # attributes and method of super_class
```

```
    # attributes and method of sub_class
```



- Execution of a *derived class* definition proceeds **the same** as for a *base class*. When the class object is constructed, the base class is remembered.
- This is used for resolving attribute references. This rule is applied **recursively** if the base class itself is derived from some other class.
- Derived classes may **override** methods of their base classes.



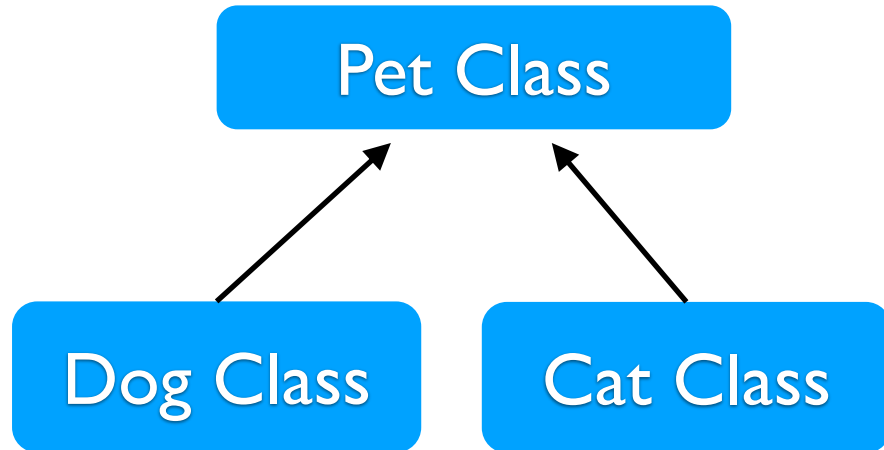
Example of Inheritance

- Define classes for **dog** and **cat**





Example of Inheritance



Pet Class:

- Attributes:
Name, Age
- Methods:
Speak, Info

Dog Subclass:

- Attributes:
Breed
- Methods:
Speak, Info

Cat Subclass:

- Attributes:
Colour
- Methods:
Speak, Info



Example of Inheritance

- First define the parent class - Pet:

```
class Pet():  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age  
  
    def speak(self):  
        print("This pet makes a sound.")  
  
    def description(self):  
        print(self.name, "is", self.age, "years old.")
```



Example of Inheritance

- Define the Dog class:

```
class Dog(Pet):  
    def __init__(self, name, age, breed):  
        super().__init__(name, age)  
        self.breed = breed  
  
    def speak(self):  
        print("Woof! Woof!")  
  
    def info(self):  
        print(self.name, "is a", self.age, "years old", self.breed)  
  
my_dog = Dog("Buddy", 5, "Golden Retriever")  
my_dog.info()      # Buddy is a 5 years old Golden Retriever  
my_dog.speak()     # Woof! Woof!
```



Example of Inheritance

- Then the Cat class:

```
class Cat(Pet):  
    def __init__(self, name, age, colour):  
        super().__init__(name, age)  
        self.colour = colour  
  
    def speak(self):  
        print("Meow!")  
  
    def info(self):  
        print(self.name, "is a", self.age, "-year-old", self.colour, "cat.")  
  
my_cat = Cat("Luna", 2, "grey")  
my_cat.info()    # Luna is a 2-year-old grey cat.  
my_cat.speak()  # Meow!
```



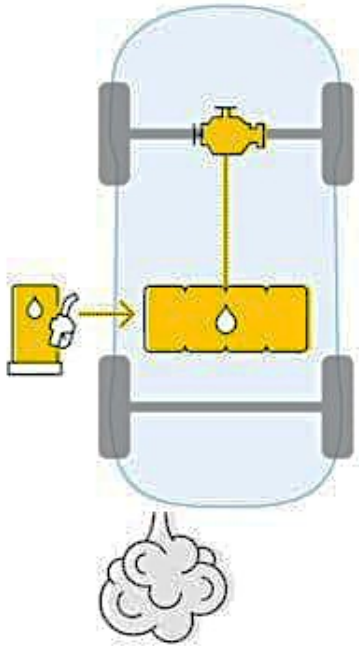
Inheritance

- Inheritance is an **is-a** relationship. That is, we use inheritance only if there exists an **is-a** relationship between two classes.
- For example,
 - **EV** is a **Car**
 - **Apple** is a **Fruit**
 - **Cat** is an **Animal**
- Here, **EV** can inherit from **Car**, **Apple** can inherit from **Fruit**, and so on.



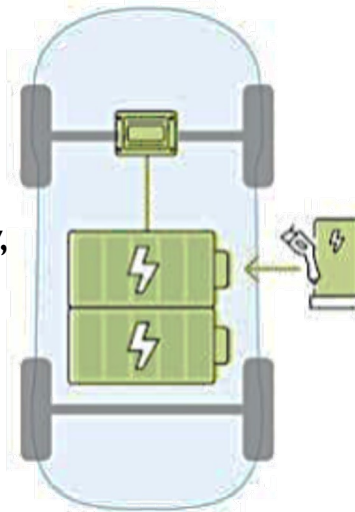
Multiple Inheritance

ICE (Internal Combustion Engine)

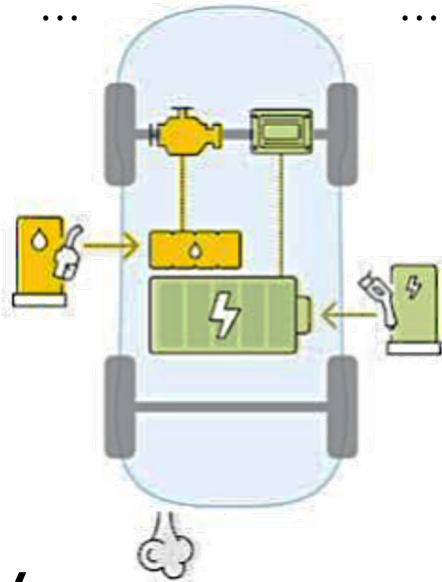


make,
model,
tank,
petrol,
fillup,
...

BEV (Battery Electric Vehicle)



make,
model,
battery,
electricity,
charge,
...



PHEV (Plug-in Hybrid Electric Vehicle)

Think about this:

A PHEV contains features of an ICE car, and also those of a BEV.

How can we define its inheritance?

Python supports a form of **multiple inheritance**.

Multiple Inheritance

- A class definition with multiple base classes looks like this:

```
class DerivedClassName(Base1, Base2, Base3):
```

```
    <statement-1>
```

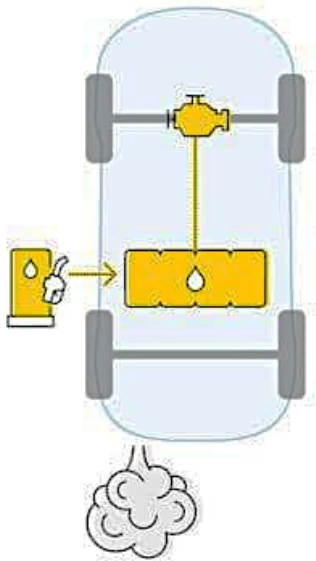
```
    ...
```

```
    <statement-N>
```

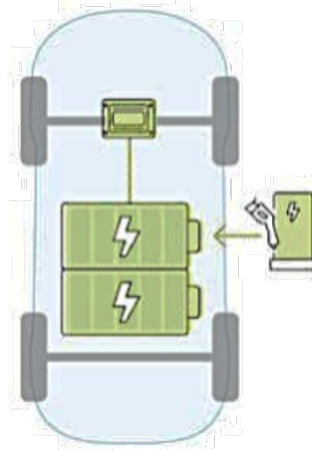
- For most purposes, in the simplest cases, you can think of the search for attributes inherited from a parent class as depth-first, left-to-right, not searching twice in the same class where there is an overlap in the hierarchy.
- If an attribute is not found in `DerivedClassName`, it is searched for in `Base1`, then (recursively) in the base classes of `Base1`, and if it was not found there, it was searched for in `Base2`, and so on.
- Dynamic ordering is necessary because all cases of multiple inheritance exhibit one or more diamond relationships (where at least one of the parent classes can be accessed through multiple paths from the bottommost class).

Multiple Inheritance

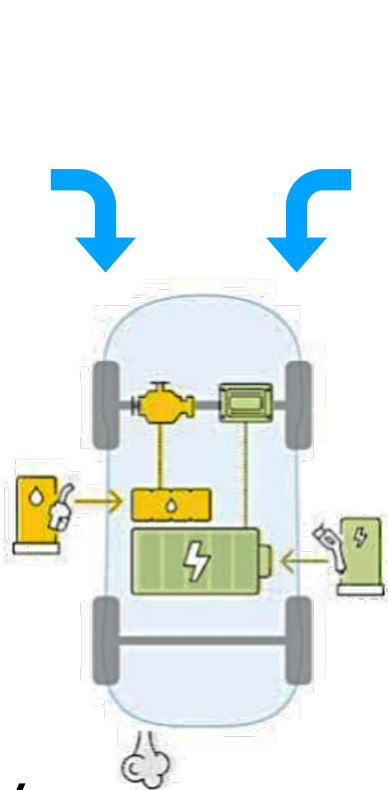
ICE (Internal Combustion Engine)



BEV (Battery Electric Vehicle)



PHEV (Plug-in Hybrid Electric Vehicle)



- In this example, if we already have the base classes ICE and BEV:

```
class ICE():
```

```
class BEV():
```

- We can define a derived class to inherit from both of them.

```
class PHEV(ICE, BEV):
```

- Then the class PHEV has all the attributes and methods of ICE and BEV.



University
of Glasgow

More on inheritance

You've already seen this,
but just to give extra insight



Recap - Inheritance

- Think of the phrase '*is a*'
- So a pigeon *is a* bird. So the base/super/parent class is the bird and the inherited/sub/child/derived class is a pigeon.
- a pigeon can access all the data types and employ all the functions of a bird.
- Be careful - for instance *flight* is not for all



Recap - Inheritance

- Not all birds fly.
- So the *base* might be a bird (wings, flapping)
- And a subclass might be a non-flightless bird (add airspeed, levitation)
- And a sub-subclass could be a pigeon. (add dirt, annoying commuters)



What Inheritance isn't

- Just flag another sort of relationship '*has a*'
- A car *has an* engine, wheels, seats, ...
- The car class does not 'inherit' in the same way we have been discussing.
- But as explained, we can make a car class with *data* which are themselves objects of class engine, ...